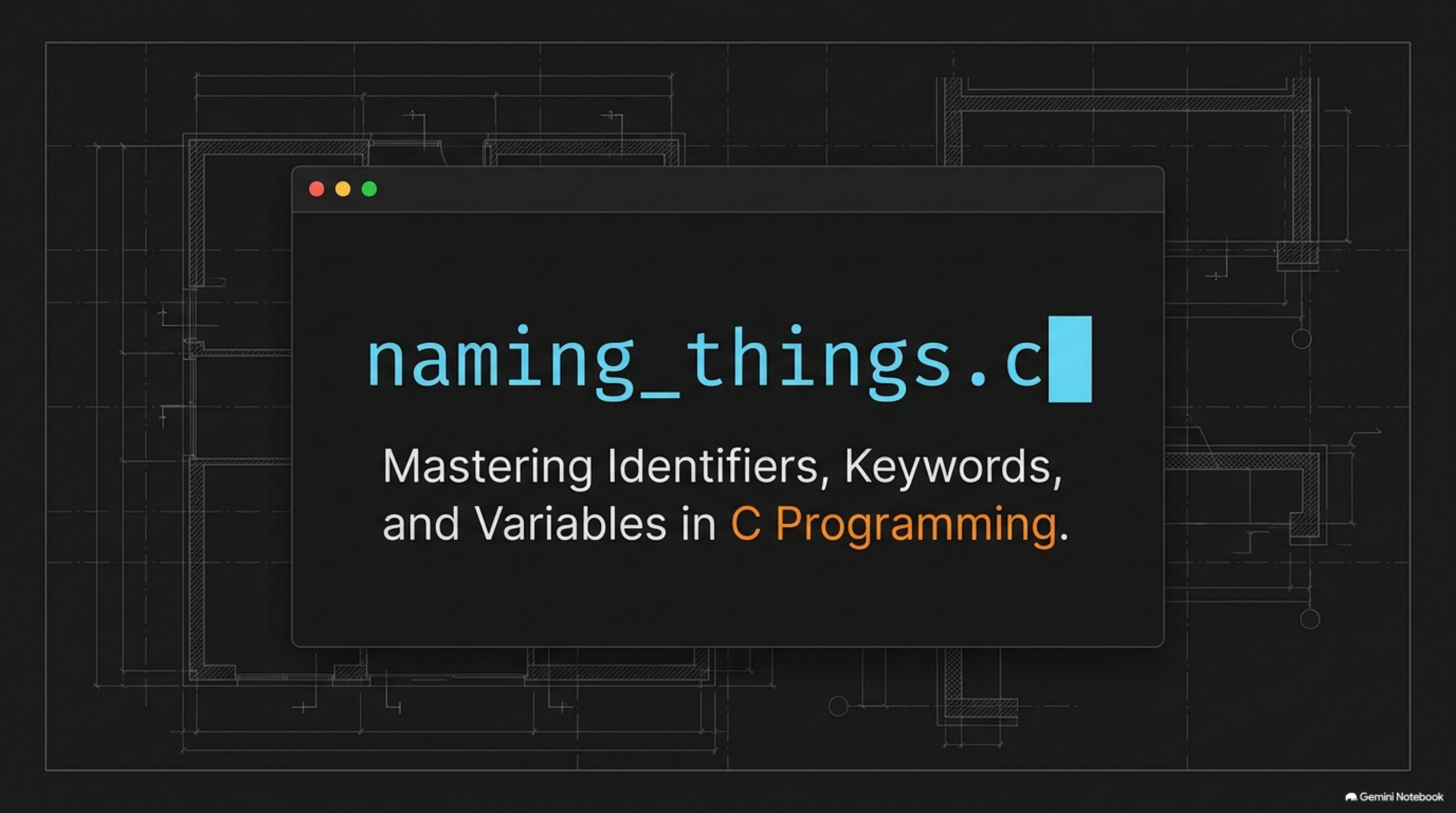


The background of the slide is a dark gray with a faint, light gray technical drawing or blueprint. It features various geometric shapes, lines, and hatching patterns, typical of architectural or engineering drawings. In the center, there is a dark gray rectangular window with a thin border and three small colored circles (red, yellow, green) in the top-left corner, resembling a standard OS window. Inside this window, the text "naming_things.c" is displayed in a light blue, monospaced font, followed by a solid light blue square.

naming_things.c

Mastering Identifiers, Keywords,
and Variables in C Programming.

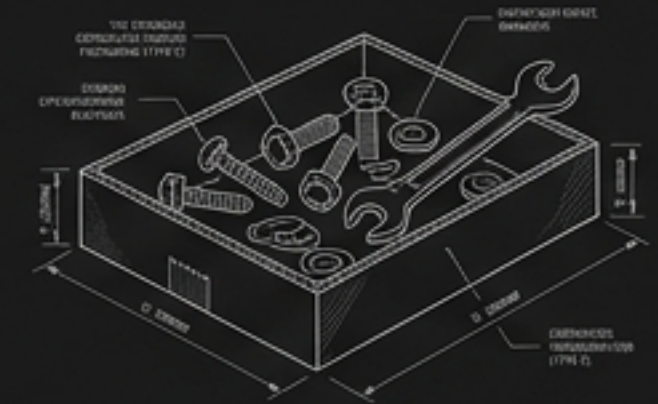
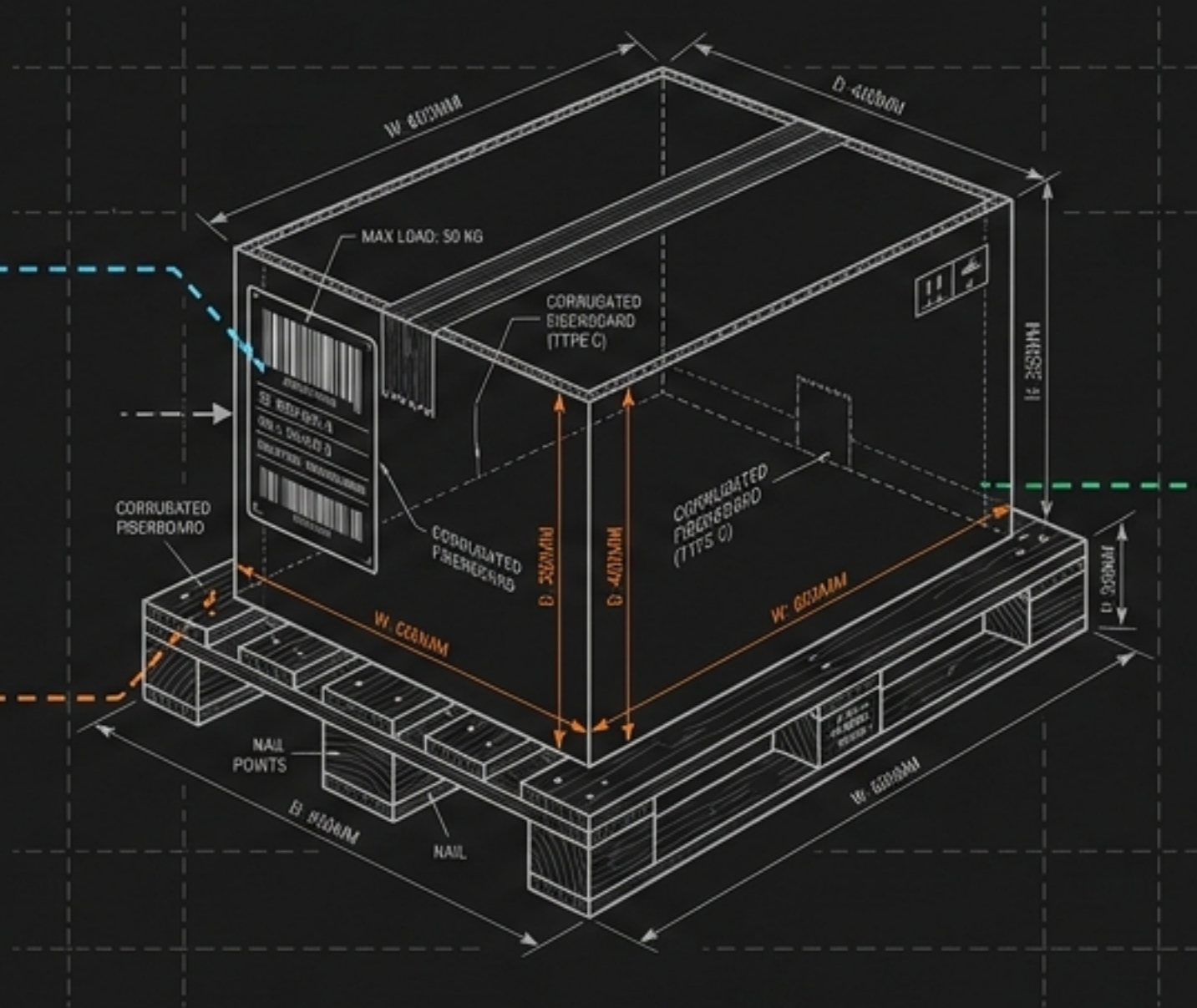
Every program is a warehouse of labeled information.

Identifiers

The name tag so workers know what's inside without opening it.

Data Types

The structural rules—a box designed for screws cannot hold water.

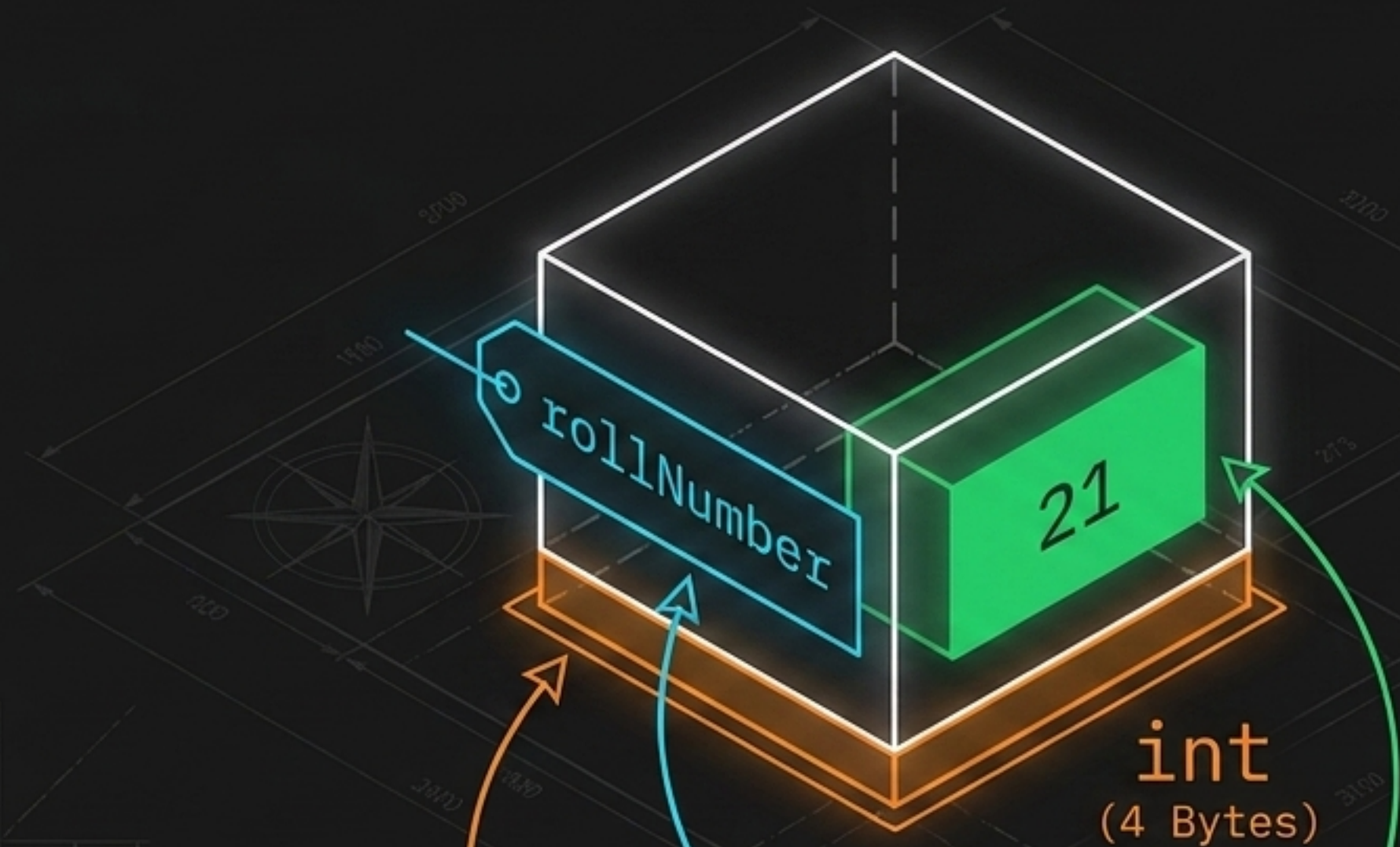


Variables

The storage space where contents can be emptied and refilled.

Before writing a single line of logic, you must master the strict rules for building and labeling these boxes.

The Anatomy of a Memory Box



```
int rollNumber = 21;
```


The strict geometry of a valid identifier.

The Valid Name ✓

totalMarks

Starts with a letter
(or underscore _)

CamelCase is
standard convention
(C is case-sensitive)

Only letters, digits, and
underscores allowed.

The Invalid Name

1st Rank

Fatal: Cannot start
with a digit.

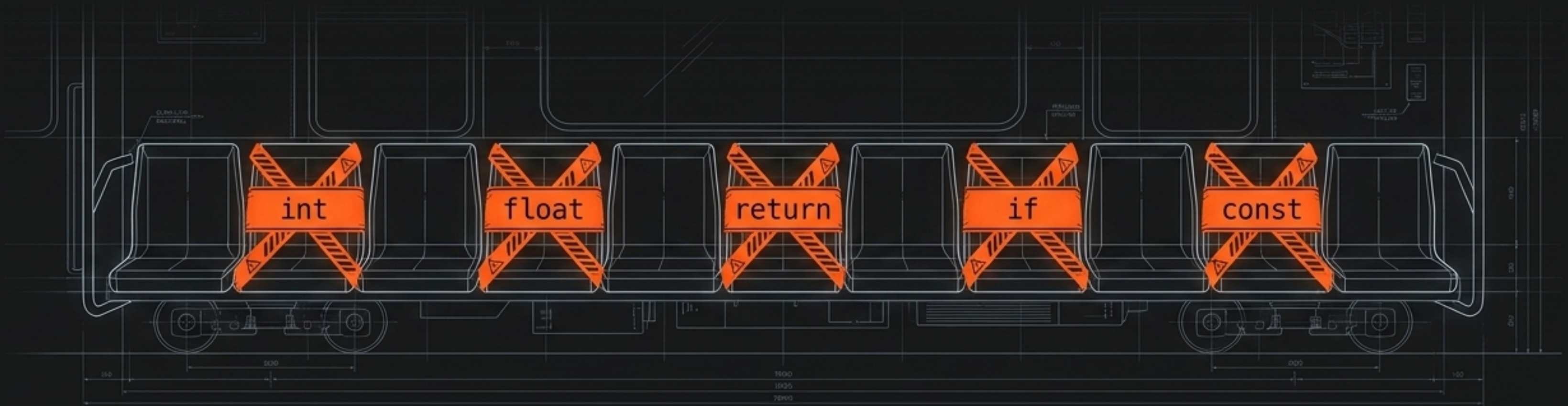
Fatal: No spaces allowed
(Compiler reads this as two
separate words).

Identifiers are like usernames. C enforces
strict discipline on allowed characters.

The Identifier Linter Matrix

Valid Identifiers ✓	Invalid Identifiers ✗
<code>_count</code> → Valid (Starts with underscore, legal but unconventional).	<code>total-marks</code> → Invalid (Hyphen - is read as a subtraction operator).
<code>student1</code> → Valid (Digits are allowed after the first character).	<code>marks%</code> → Invalid (Special symbols are prohibited).
<code>Marks & marks</code> → Valid (Treated as two completely separate variables due to case-sensitivity).	<code>float</code> → Invalid (Reserved keyword—C is already using this word).

Keywords are permanently reserved seats.



Standard C defines exactly 32 keywords. These words have a fixed, structural meaning to the compiler. You cannot use them as variable names, no matter how convenient it seems.

```
int float = 10;
```

```
Compile Error: 'float' is already taken.
```


Structural scaffolding in action.

Reserves memory for a whole number.

```
int attendance = 78;
```

```
if (attendance >= 75) {  
    printf("Eligible");  
} else {  
    printf("Not eligible");  
}
```

```
return 0;
```

Forms a decision structure (never runs both).

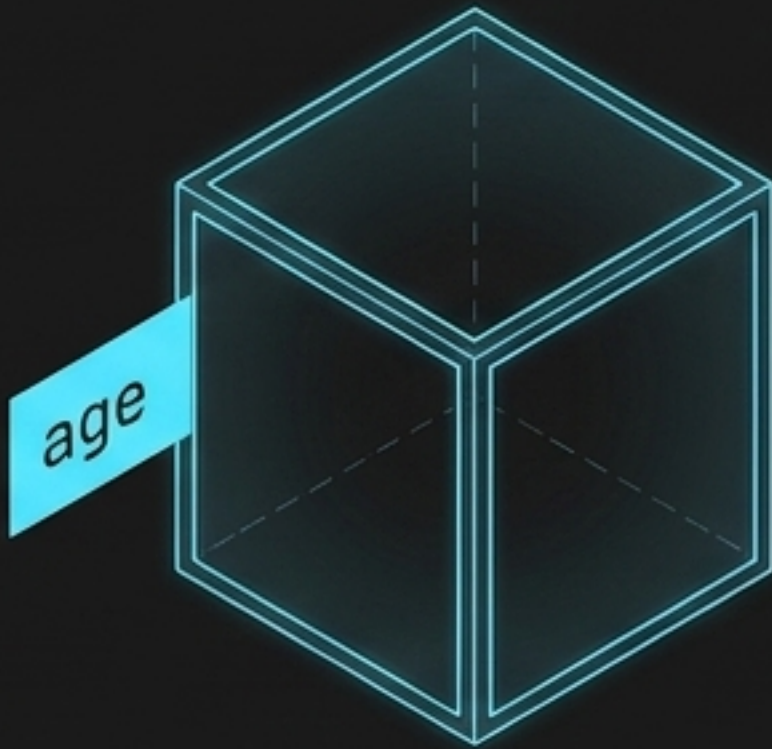
Ends the function and reports success to the OS.

The lifecycle of a memory box.

Step 1: Declaration

(Building the empty box)

```
int age;
```



Step 2: Initialization

(Filling the box for the first time)

```
age = 20;
```



Step 3: Combined

(The efficient one-liner)

```
int age = 20;
```

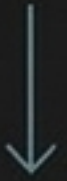


The Data Type Dimensions

Data Type	What it Stores	Memory Size (Bytes)	Format Specifier
<code>int</code>	Whole Numbers (42)	4 bytes	<code>%d</code>
<code>float</code>	Decimal Numbers (99.50)	4 bytes	<code>%f</code>
<code>char</code>	Single Character ('A') *Note: Single quotes required.*	1 byte	<code>%c</code>
<code>double</code>	High-Precision Decimals (3.14159265)	8 bytes	<code>%lf</code>

Variables in motion: The Mutation Stack.

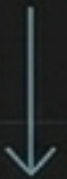
```
int score = 0;
```



```
score = 10;
```



```
score = score + 5;
```

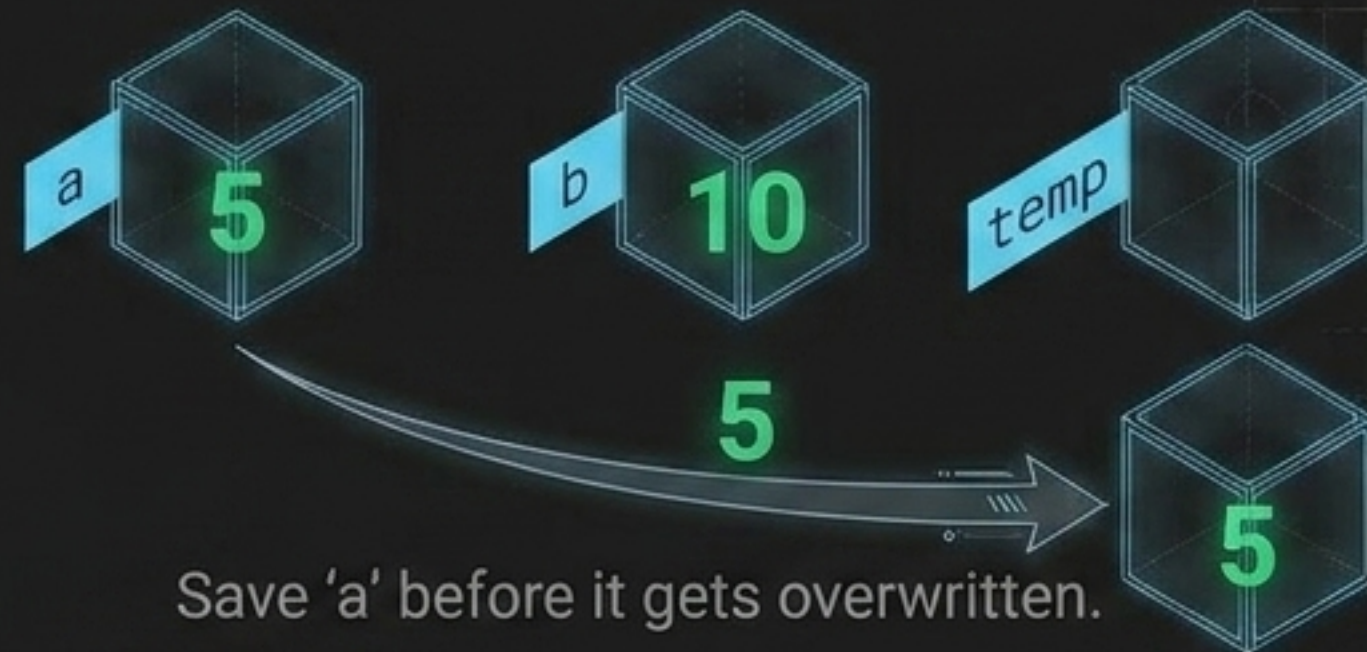


```
score += 20; [Compound Assignment Operator]
```

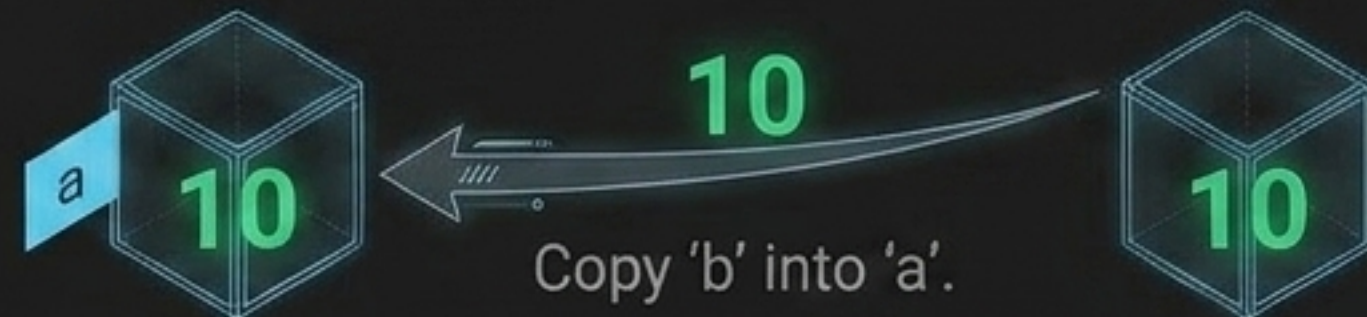


The logic of swapping data safely.

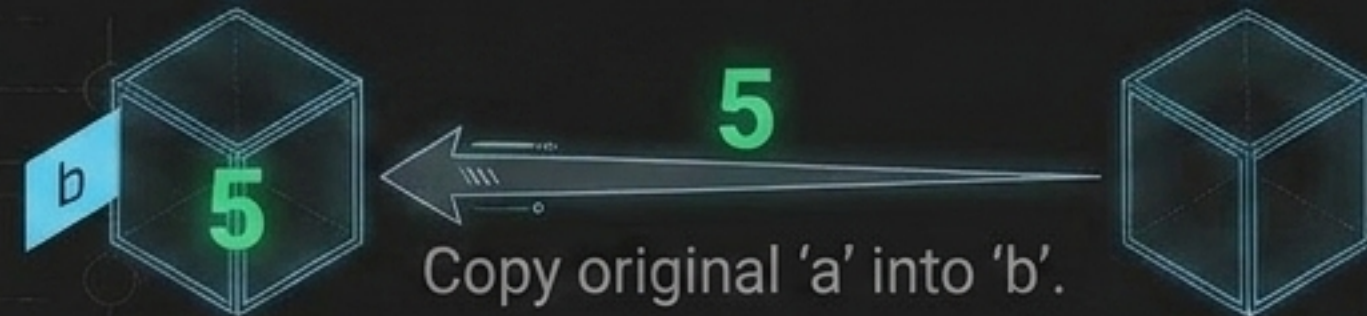
Step 1. `temp = a;`



Step 2. `a = b;`



Step 3. `b = temp;`

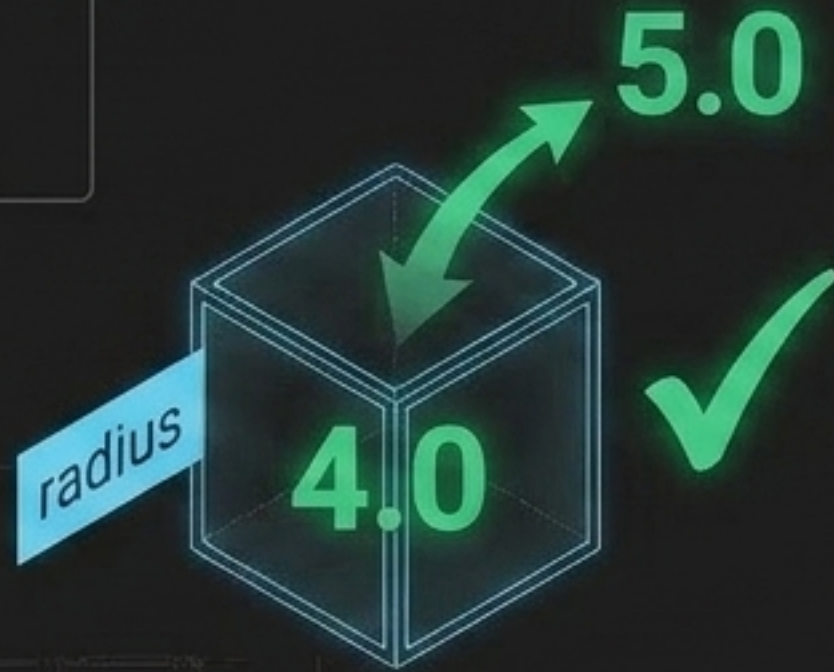


Warning: Doing `a = b;` `b = a;` directly destroys the original value of 'a' immediately!

Constants: Variables that refuse to change.

The Variable

```
float radius = 4.0;  
radius = 5.0;
```



The Constant

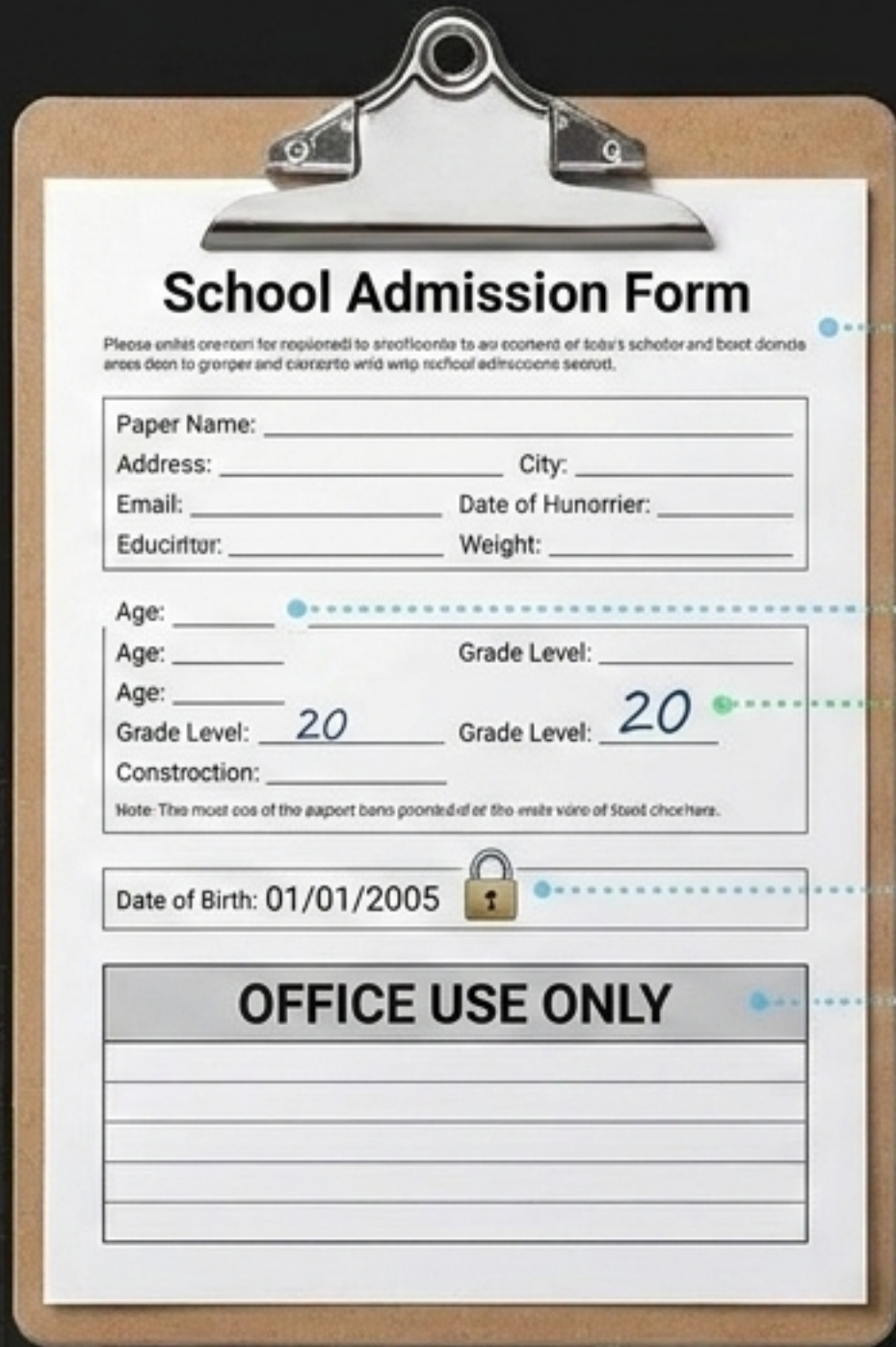
```
const float PI = 3.14159;  
PI = 3.14;
```



Compile Error: assignment of read-only variable 'PI'.

Convention Tip: Constants are written in ALL_CAPS to instantly signal they are locked.

Synthesizing the concepts: The Admission Form.



School Admission Form

Please enter information for registration to attend school to all content of this school and best don't do it
areas don't to proper and correct with with school education secret.

Paper Name: _____
Address: _____ City: _____
Email: _____ Date of Birth: _____
Education: _____ Weight: _____

Age: _____
Age: _____ Grade Level: _____
Age: _____
Grade Level: 20 Grade Level: 20
Construction: _____

Note: This must cos of the report bars pointed at the write side of the school.

Date of Birth: 01/01/2005 

OFFICE USE ONLY

Identifiers (The names of the storage spaces)

Variables (The empty memory waiting for data)

Values (The data stored inside)

Constants (const)

Keywords (Reserved structural words)

Common compiler collisions.

The Mistake (Red Code)

```
int 2ndValue;
```

```
int char;
```

```
float price = 10.5
```

```
printf("%d", 99.50);
```

Why it Fails

Identifiers cannot start with a digit.

'char' is a reserved keyword; you cannot use it as a name.

Missing semicolon; the compiler doesn't know where the statement ends.

Using %d (integer specifier) for a float value prints broken garbage data.

Pro-Tip: If your compiler says "expected identifier before int", you likely tried to use a reserved keyword as a variable name.

The Memory Master's Cheat Sheet.

Identifier	Keyword	Variable	Constant	Compound Assignment
A user-defined name (<code>`totalMarks`</code>). Must start with letter/underscore. No spaces. Case-sensitive.	32 reserved words with fixed meaning (<code>`int`</code>, <code>`return`</code>). Cannot be used as identifiers.	A named, typed box where values can change over time. <pre>int age = 20;</pre>	A locked variable that cannot change post-initialization. <pre>const float PI = 3.14;</pre>	Shorthand for updating based on the current value. <pre>score += 5;</pre>

Next up: Deep dive into C data types, memory ranges, and choosing the exact right box for your data.